23CSE455 Design Patterns

Case Study Report

Team: B10

Case study: Agriculture management (sensor-driven irrigation and alerts); Python / Flask; "Design patterns in Mycodo"

Application: Mycodo v8.17.0 – open-source environmental monitoring and regulation system (GPL-3.0), used as the agriculture management application

Date: 2026-10-25

Submission repository: 23CSE455_DP_CaseStudy (team folder `B10/`)

Students

Roll number	Student name	Batch
AM.SC.U4AIE23038	Johan Rajesh	AIE
AM.SC.U4AIE23003	Aadarsh Jayan Nair	AIE
AM.SC.U4AIE23029	Dano Aby Alex	AIE
AM.SC.U4AIE23140	P Arya	AIE

1 Application and architecture

Original source: `https://github.com/kizniche/Mycodo.git`, release v8.17.0, baseline commit `ee70a111c622e2a81026fd5c1553a948acf8161e`, accessed 2026-10-11, GPL-3.0. Saved unchanged in `data/code/`.

Environment: Python 3.12.7 with Flask 3.1.2, Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.0.45, Flask-Login 0.6.3, marshmallow-sqlalchemy 1.4.2, Pyro5 5.16, bcrypt 5.0.0, requests 2.32.5 (pins from `install/requirements.txt`); SQLite settings database and InfluxDB time-series store (mocked in our tests). Part 3: OpenJDK 21.0.2, Node.js v23.11.0, MinGW g++ 6.3.0 (`-std=c++14`). Tools: pytest 9.0.2, lizard 1.24.1. Windows 11.

Scope: 24 files under `data/code/mycodo/` that form the agriculture path: sensor inputs (soil moisture, weather), controller threads, irrigation outputs (pump, relay), the bang-bang irrigation function, setpoint schedules, module loading, conditional alerts and actions, the UI-to-daemon RPC client and the user/role models. Excluded: the Flask UI (`mycodo_flask/`), the daemon server, ~400 other device modules and the migrations, because they are outside the irrigation scenario or repeat the same base-class collaborations. The file list is in `README.md`.

Architecture: Figure 1. Mycodo is a daemon of controller threads plus a Flask web UI. Every daemon-side module (input, output, function, action) extends `AbstractBaseController`. Event flow for one soil reading: the daemon starts an `InputController` thread; `AbstractController.run` calls `initialize_variables`, which loads the input module through `load_module_from_file`; `loop` periodically calls `update_measure` → `AbstractInput.next/read` → `adafruit_i2c_soil.InputModule.get_measurement` → the Seesaw I^2C driver; the values are rescaled, converted and written to InfluxDB. The bang-bang function reads the last value and switches the pump output through `DaemonControl` (Pyro5 RPC).

Run instructions: `README.md` (team folder). Part 2: `cd Part2/code; python -m pytest -p no:cacheprovider -v mycodo/tests/case_study` → 83 passed. Part 3: per-language commands in each `README.md`. All suites: `python -B data/test_result/run_all_tests.py` → Part 2 83/83, 3 demos OK, Part 3 32/32 (run 2026-10-25, logs in `data/test_result/`).

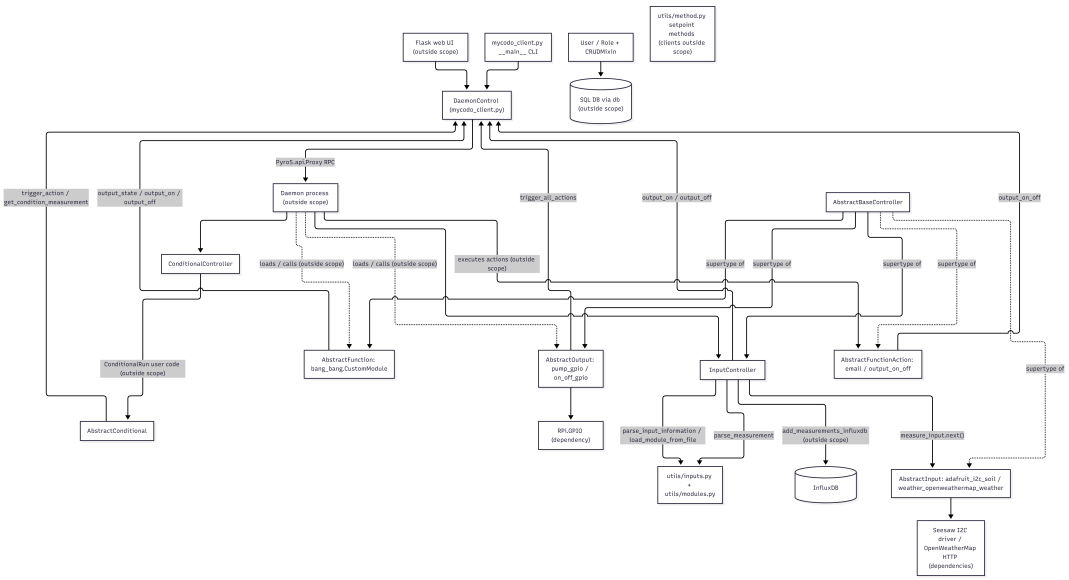


Figure 1. Architecture of the inspected scope (source: `Part1/pattern_architecture/00_architecture.mmd`).

2 Part 1 Pattern discovery

Method: An LLM (Claude Opus 5.5) read the 24 scope files with a fixed prompt (`Part1/llm/part1_prompt_Johan.txt`) and proposed candidates for GoF and PoEAA patterns, each with roles mapped to real symbols and line ranges. A candidate was accepted only if the collaboration (calls, inheritance, delegation) is visible in the code; names or annotations alone were not enough. A script checked that every `path:start-end` exists and lies inside its file (112 references). The team compared the result with an independent member list (29 rows: renamed or split versions of the same instances, candidates we keep as rejected or uncertain, and rows outside the 24-file scope) and kept the 13 instances below.

2.1 Totals at the baseline

Confirmed pattern types: 8: Template Method, Strategy, Adapter, Proxy, Command, Plugin, Layer Supertype, Active Record.

Confirmed pattern instances: 13 independent collaborations (Template Method ×6, one of each other type).

Concrete participant classes: 18: `adafruit_i2c_soil.InputModule`, `weather_openweathermap_weather.InputModule`, `pump_gpio.OutputModule`, `on_off_gpio.OutputModule`, `bang_bang.CustomModule`, `email.ActionModule`, `output_on_off.ActionModule`, `InputController`, `ConditionalController`, `DateMethod`, `DailyMethod`, `DailySineMethod`, `DailyBezierMethod`, `DurationMethod`, `CascadeMethod`, `DaemonControl`, `User`, `Role` (23 if the five abstract Layer Supertype members are included).

Unique participant symbols: 85 = 79 in scope (27 classes, 41 methods, 7 module functions, 4 other) + 6 external (`threading.Thread`, `Seesaw`, `requests.get`, `Pyro5.api.Proxy`, `send_email`, `db.Model`); shared symbols counted once.

Rejected and uncertain: 23 rejected, 5 uncertain (observation scope: the 24 files only). See the table below.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Template Method	Sensor Measurement Acquisition; Module Initialization with Retry; Controller Thread Lifecycle; Output Actuation (irrigation switching); Formula Setpoint Plot Sampling; Date/Daily Setpoint Interpolation	6	14
Strategy	Setpoint Schedule Calculation	1	6
Adapter	Sensor Driver Adaptation	1	2
Proxy	Daemon RPC Client	1	1
Command	Automation/Alert Actions	1	2
Plugin	Input/Output Module Loading	1	4
Layer Supertype	Daemon Module Base	1	5
Active Record	User and Role Persistence	1	2

Concrete classes are counted per instance (a class can take part in several instances, so the column sum is larger than the 18 distinct classes in 2.1). For Layer Supertype the five abstract layer members are counted.

Rejected and uncertain candidates

Candidate	Result	Reason
Observer – output trigger notification	Uncertain	Subscribers are DB rows selected by query; no attach/detach and no observer objects; delivery delegated to the out-of-scope daemon.
Composite – cascade setpoint methods	Uncertain	Recursive `calculate_setpoint`, but children are reloaded by id, no child management, `isinstance` breaks transparency.
Mediator – daemon as coordinator	Uncertain	All modules talk through `DaemonControl`, but the daemon is outside scope.
Plugin / Template Method – user Conditional code	Uncertain	`ConditionalRun` is generated outside scope; inheritance from `AbstractConditional` cannot be verified.
Adapter – outputs over RPi.GPIO	Uncertain	Procedural GPIO module; the translating methods are already the Output Actuation primitives.
Singleton	Rejected	DaemonControl() is constructed in 6 places; nothing controls instantiation.
Factory Method	Rejected	`create_method_handler` is a plain function; no creator hierarchy (counted inside Strategy / Plugin).
Prototype	Rejected	`clone_model` is a reflective helper with no caller in scope.
Facade	Rejected	`DaemonControl` forwards 1:1 to one remote object (counted as Proxy).
Decorator (baseline)	Rejected	Only Python `@property` / `@staticmethod` language decorators.
Iterator	Rejected	`__next__` returns a fresh reading; no aggregate is traversed.
Repository	Rejected	ORM queries are inline in callers; no repository abstraction.
MVC	Rejected	Only models are in scope.
Chain of Responsibility	Rejected	Every action handles the request; no successor link.
State	Rejected	Output state is dicts/flags and an if/elif on a string.
Strategy – bang-bang direction / rescale method	Rejected	Conditional branches on strings, not objects.
Strategy – InputController.measure_input	Rejected	Already counted in Template Method, Adapter and Plugin.
Template Method on plain overrides	Rejected	Constructors / full overrides with no base skeleton.

Candidate	Result	Reason
Command – CLI dispatch	Rejected	If/elif over argparse flags; no request objects.
Null Object – AbstractMethod fallback	Rejected	Part of the Strategy collaboration.
Broker	Rejected	Same collaboration as Proxy; broker side outside scope.
Unit of Work	Rejected	`session_scope` is defined outside scope.
Virtual Proxy / Lazy Load	Rejected	`measurements` lazily calls `read()` on the same object.
Framework hook – UserMixin	Rejected	No hooks overridden in scope.
DTO / Serializer – UserSchema	Rejected	Declarative framework configuration only.
Value Object – Measurement	Rejected	Mutable holder used inside one class.
Retry	Rejected	Resilience idiom; its hook collaboration is Template Method 2.
GRASP principles	Rejected	Excluded: every case is already covered by a GoF / architectural instance.

2.3 One record for each pattern instance

2.3.1 Sensor Measurement Acquisition

Instance name: Sensor Measurement Acquisition

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
AbstractClass	`AbstractInput`	`data/code/mycodo/inputs/base_input.py:18-271`
TemplateMethod	`AbstractInput.read`	`data/code/mycodo/inputs/base_input.py:115-146`
TemplateMethodEntry	`AbstractInput.next`	`data/code/mycodo/inputs/base_input.py:88-92`
PrimitiveOperation	`AbstractInput.get_measurement`	`data/code/mycodo/inputs/base_input.py:108-113`
ConcreteClass	`adafruit_i2c_soil.InputModule`	`data/code/mycodo/inputs/adafruit_i2c_soil.py:50-84`
ConcreteClass	`weather_openweathermap_weather.InputModule`	`data/code/mycodo/inputs/weather_openweathermap_weather.py:82-146`
Client	`InputController.update_measure`	`data/code/mycodo/controllers/controller_input.py:345-393`

Evidence: Fixed sensor-reading algorithm with a device-specific step: read() resets the cache, calls the abstract get_measurement() (base_input.py:123), classifies TimeoutError/IOError/other errors and returns 1 on failure (126-146); next() turns failure into StopIteration (90-91). The soil sensor (adafruit_i2c_soil.py:71-84) and weather input (weather_openweathermap_weather.py:102-146) override only get_measurement; InputController.update_measure calls measure_input.next() (controller_input.py:361).

Diagram: `Part1/pattern_architecture/01_template_method_sensor_measurement_acquisition.mmd` (editable Mermaid) and `.png` (Figure 2).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and

rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

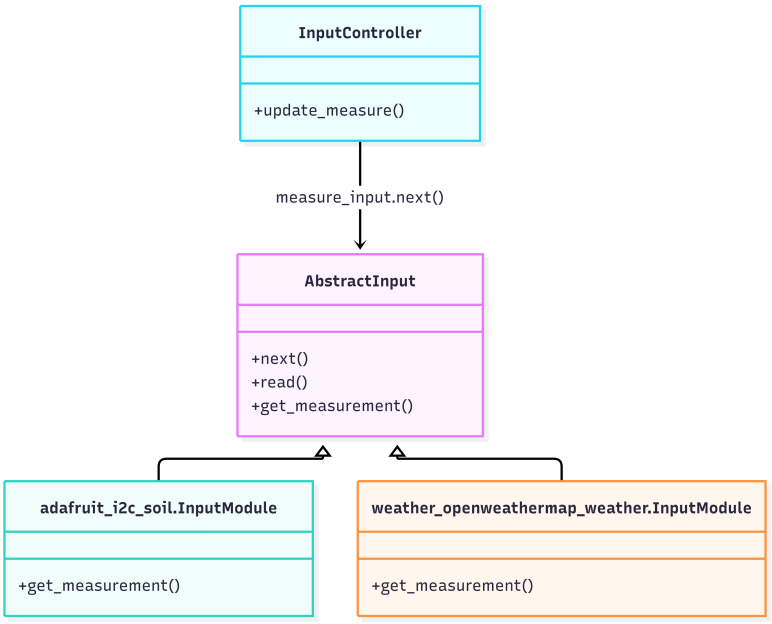


Figure 2. Template Method – Sensor Measurement Acquisition.

2.3.2 Module Initialization with Retry

Instance name: Module Initialization with Retry

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
AbstractClass	`AbstractBaseController`	`data/code/mycodo/abstract_base_controller.py:35-543`
TemplateMethod	`AbstractBaseController.try_initialize`	`data/code/mycodo/abstract_base_controller.py:52-65`
PrimitiveOperation	`AbstractBaseController.initialize` (default no-op hook)	`data/code/mycodo/abstract_base_controller.py:49-50`
PrimitiveOperation	`AbstractInput.initialize` (abstract redeclaration)	`data/code/mycodo/inputs/base_input.py:101-106`
PrimitiveOperation	`AbstractFunction.initialize` (abstract redeclaration)	`data/code/mycodo/functions/base_function.py:57-62`
PrimitiveOperation	`AbstractFunctionAction.initialize` (abstract redeclaration)	`data/code/mycodo/actions/base_action.py:70-75`
ConcreteClass	`adafruit_i2c_soil.InputModule`	`data/code/mycodo/inputs/adafruit_i2c_soil.py:50-84`
ConcreteClass	`weather_openweathermap_weather.InputModule`	`data/code/mycodo/inputs/weather_openweathermap_weather.py:82-146`
ConcreteClass	`bang_bang.CustomModule`	`data/code/mycodo/functions/bang_bang.py:110-213`
ConcreteClass	`email.ActionModule`	`data/code/mycodo/actions/email.py:41-98`

Pattern role	Actual class / function	File and line range
ConcreteClass	`output_on_off.ActionModule`	`data/code/mycodo/actions/output_on_off.py:68-139`

Evidence: Retry skeleton for fragile hardware/service set-up: try_initialize loops tries times calling the polymorphic self.initialize() and sleeps wait_sec on exceptions without propagating them (abstract_base_controller.py:54-65). Concrete modules call self.try_initialize() from their constructors (adafruit_i2c_soil.py:58; weather_openweathermap_weather.py:94; bang_bang.py:142; email.py:54; output_on_off.py:84) and override only initialize().

Diagram: `Part1/pattern_architecture/02_template_method_module_initialization_with_retry.mmd` (editable Mermaid) and `.png` (Figure 3).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

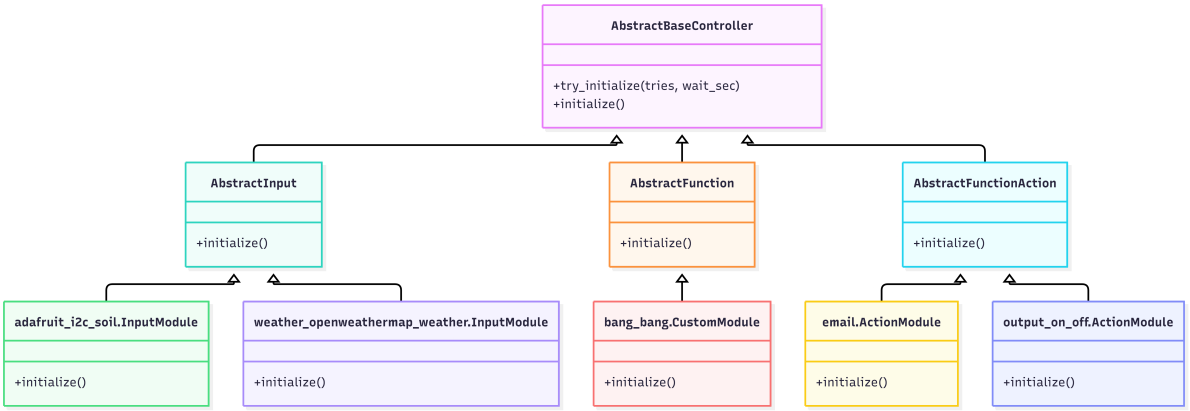


Figure 3. Template Method – Module Initialization with Retry.

2.3.3 Controller Thread Lifecycle

Instance name: Controller Thread Lifecycle

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
AbstractClass	`AbstractController`	`data/code/mycodo/controllers/base_controller.py:20-129`
TemplateMethod	`AbstractController.run (overrides threading.Thread.run)`	`data/code/mycodo/controllers/base_controller.py:69-99`
TemplateMethod	`AbstractController.stop_controller`	`data/code/mycodo/controllers/base_controller.py:104-107`
PrimitiveOperation	`AbstractController.initialize_variables`	`data/code/mycodo/controllers/base_controller.py:45-49`
PrimitiveOperation	`AbstractController.loop`	`data/code/mycodo/controllers/base_controller.py:51-55`
Hook	`AbstractController.run_finally`	`data/code/mycodo/controllers/base_controller.py:57-59`

Pattern role	Actual class / function	File and line range
Hook	`AbstractController.pre_stop`	`data/code/mycodo/controllers/base_controller.py:61-63`
ConcreteClass	`InputController`	`data/code/mycodo/controllers/controller_input.py:65-462`
ConcreteClass	`ConditionalController`	`data/code/mycodo/controllers/controller_conditional.py:41-197`
FrameworkCaller	`threading.Thread (Python standard library, defined outside scope)`	`data/code/mycodo/controllers/controller_input.py:65-70`

Evidence: Invariant controller-thread lifecycle: run() calls initialize_variables() (base_controller.py:72), loops loop() while running with per-iteration error isolation and sleep(sample_rate) (79-87) and always calls run_finally() (93); stop_controller() calls pre_stop() (106). InputController and ConditionalController inherit (AbstractController, threading.Thread) so run() is the application override of the stdlib Thread.run hook; they override loop/initialize_variables (controller_input.py:129-343; controller_conditional.py:73-147).

Diagram: `Part1/pattern_architecture/03_template_method_controller_thread_lifecycle.mmd` (editable Mermaid) and `.png` (Figure 4).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

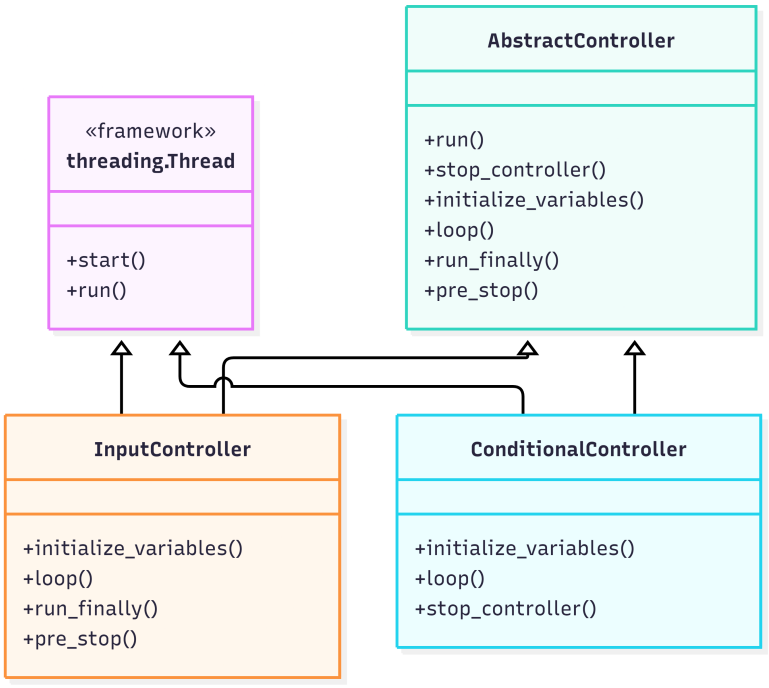


Figure 4. Template Method – Controller Thread Lifecycle.

2.3.4 Output Actuation (irrigation switching)

Instance name: Output Actuation (irrigation switching)

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function
AbstractClass	`AbstractOutput`
TemplateMethod	`AbstractOutput.output_on_off`
PrimitiveOperation	`AbstractOutput.output_switch`
PrimitiveOperation	`AbstractOutput.is_on`
PrimitiveOperation	`AbstractOutput.is_setup`
ConcreteClass	`pump_gpio.OutputModule`
ConcreteClass	`on_off_gpio.OutputModule`
Client	`defined outside scope`

Source locations: `data/code/mycodo/outputs/base_output.py:30-641`; `data/code/mycodo/outputs/base_output.py:164-455`; `data/code/mycodo/outputs/base_output.py:81-86`; `data/code/mycodo/outputs/base_output.py:88-93`; `data/code/mycodo/outputs/base_output.py:95-100`; `data/code/mycodo/outputs/pump_gpio.py:145-353`; `data/code/mycodo/outputs/on_off_gpio.py:113-200`

Evidence: Shared irrigation switching policy (state normalisation, channel/setup validation, min-off lock-out, duration bookkeeping, duration logging, trigger firing) in output_on_off, which calls the abstract is_on (base_output.py:209), is_setup (218) and output_switch (258, 343, 378, 390) and finally check_triggers (450). The peristaltic pump (pump_gpio.py:258-353) and GPIO relay (on_off_gpio.py:168-191) override only these primitive operations.

Diagram: `Part1/pattern_architecture/04_template_method_output_actuation.mmd` (editable Mermaid) and `.png` (Figure 5).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

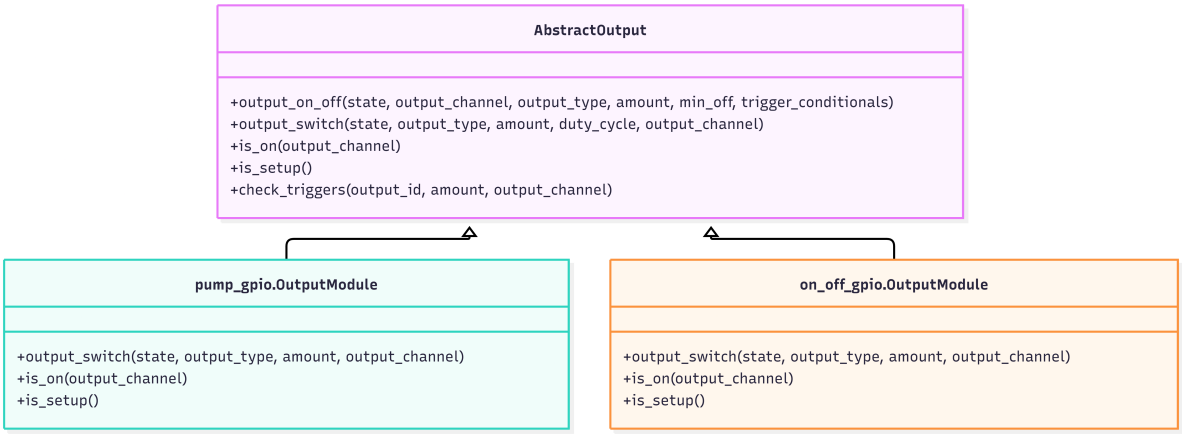


Figure 5. Template Method – Output Actuation (irrigation switching).

2.3.5 Formula Setpoint Plot Sampling

Instance name: Formula Setpoint Plot Sampling

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
AbstractClass	`AbstractDailyFormulaMethod`	`data/code/mycodo/utils/method.py:199-217`
TemplateMethod	`AbstractDailyFormulaMethod.get_plot`	`data/code/mycodo/utils/method.py:205-217`
PrimitiveOperation	`AbstractMethod.calculate_setpoint`	`data/code/mycodo/utils/method.py:57-64`
ConcreteClass	`DailySineMethod`	`data/code/mycodo/utils/method.py:220-239`
ConcreteClass	`DailyBezierMethod`	`data/code/mycodo/utils/method.py:242-258`

Evidence: get_plot samples max_points_x instants across a day and calls the polymorphic self.calculate_setpoint(now) for each, appending only non-ended points (method.py:210-215); DailySineMethod (227-239) and DailyBezierMethod (243-258) override only calculate_setpoint, delegating to sine_wave_y_out / bezier_curve_y_out.

Diagram: `Part1/pattern_architecture/05_template_method_formula_setpoint_plot_sampling.mmd` (editable Mermaid) and `.png` (Figure 6).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

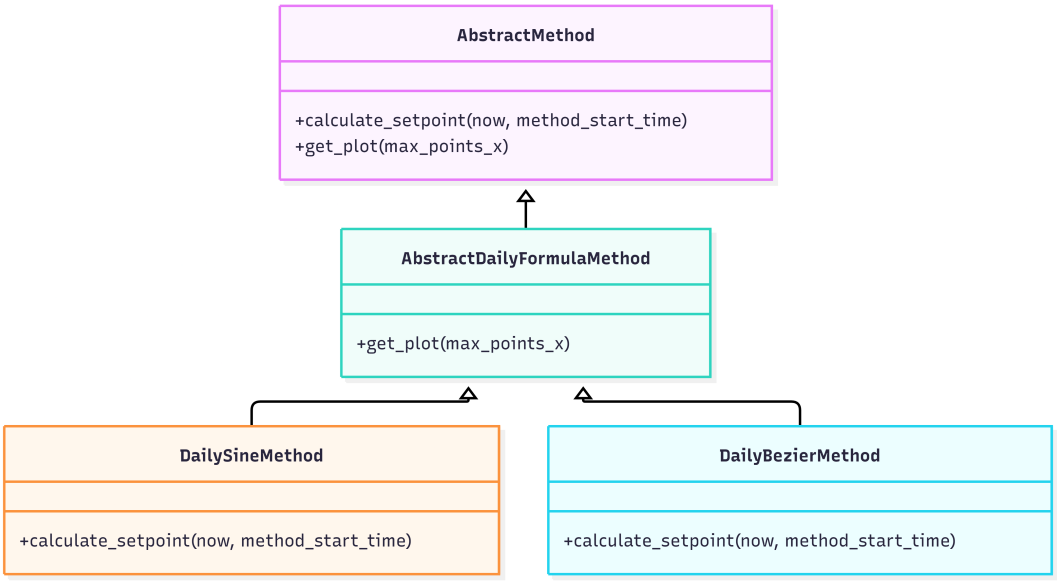


Figure 6. Template Method – Formula Setpoint Plot Sampling.

2.3.6 Date/Daily Setpoint Interpolation

Instance name: Date/Daily Setpoint Interpolation

Pattern type and category: Template Method – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
AbstractClass	`DateMethod (also a ConcreteClass providing the default hook)`	`data/code/mycodo/utils/method.py:75-162`
TemplateMethod	`DateMethod.calculate_setpoint`	`data/code/mycodo/utils/method.py:87-135`
Hook	`DateMethod.ignore_date`	`data/code/mycodo/utils/method.py:81-85`
ConcreteClass	`DailyMethod`	`data/code/mycodo/utils/method.py:165-196`

Evidence: DateMethod.calculate_setpoint interpolates the setpoint between row start/end and consults the hook self.ignore_date() to choose time parsing and logging (method.py:90-99, 119); DailyMethod overrides only the hook to return True (172-176) and thereby reuses the whole interpolation as a daily repeating schedule.

Diagram: `Part1/pattern_architecture/06_template_method_date_daily_setpoint_interpolation.mmd` (editable Mermaid) and `.png` (Figure 7).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Strategy was considered and rejected: the variation is supplied by subclass overrides called from a fixed base-class skeleton, not by an exchangeable object.

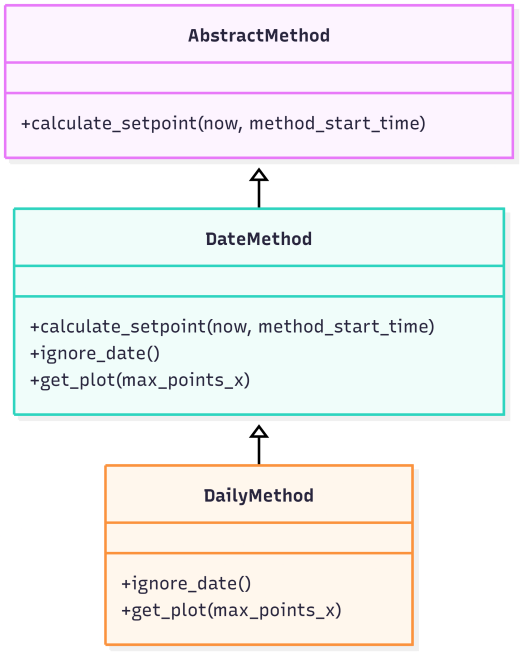


Figure 7. Template Method – Date/Daily Setpoint Interpolation.

2.3.7 Setpoint Schedule Calculation

Instance name: Setpoint Schedule Calculation

Pattern type and category: Strategy – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function	File and line range
Strategy	`AbstractMethod`	`data/code/mycodo/utils/method.py:22-72`
StrategyOperation	`AbstractMethod.calculate_setpoint`	`data/code/mycodo/utils/method.py:57-64`
ConcreteStrategy	`DateMethod`	`data/code/mycodo/utils/method.py:75-162`
ConcreteStrategy	`DailyMethod`	`data/code/mycodo/utils/method.py:165-196`
ConcreteStrategy	`DailySineMethod`	`data/code/mycodo/utils/method.py:220-239`
ConcreteStrategy	`DailyBezierMethod`	`data/code/mycodo/utils/method.py:242-258`
ConcreteStrategy	`DurationMethod`	`data/code/mycodo/utils/method.py:261-378`
ConcreteStrategy	`CascadeMethod`	`data/code/mycodo/utils/method.py:381-428`
StrategySelector	`create_method_handler`	`data/code/mycodo/utils/method.py:431-441`
StrategySelector	`load_method_handler`	`data/code/mycodo/utils/method.py:444-455`
Context	`CascadeMethod.calculate_setpoint (in-scope client, other contexts defined outside scope)`	`data/code/mycodo/utils/method.py:383-428`

Evidence: Interchangeable setpoint-schedule algorithms selected at run time from the stored method_type: create_method_handler maps the type name to an AbstractMethod subclass and instantiates it, falling back to the neutral AbstractMethod (method.py:436-441); CascadeMethod obtains linked strategies via load_method_handler and calls calculate_setpoint on them without knowing their class (402, 413-417); DurationMethod also overrides determine_end_time (337-349).

Diagram: `Part1/pattern_architecture/07_strategy_setpoint_schedule_calculation.mmd` (editable Mermaid) and `.png` (Figure 8).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Factory Method and Null Object were rejected as separate instances (the selector and fallback belong to this collaboration); Composite for CascadeMethod is uncertain.

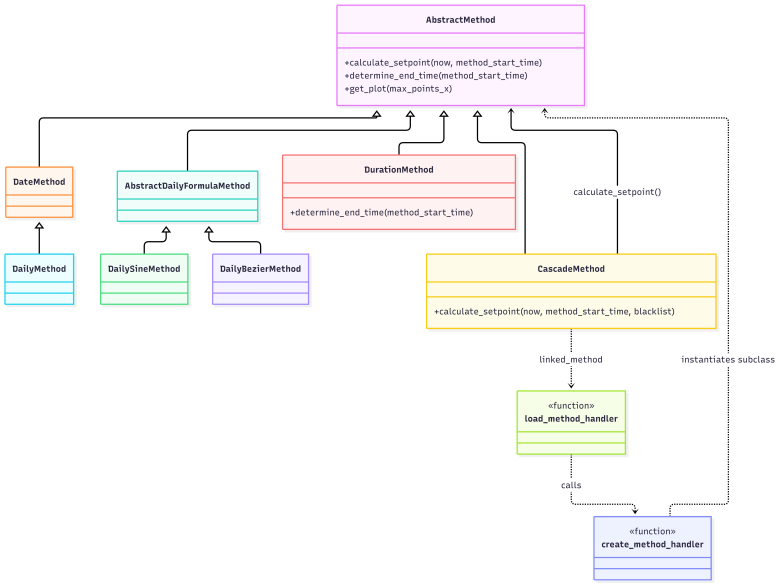


Figure 8. Strategy – Setpoint Schedule Calculation.

2.3.8 Sensor Driver Adaptation

Instance name: Sensor Driver Adaptation

Pattern type and category: Adapter – GoF Structural

Purpose and origin: application code wrapping a dependency (GoF Structural).

Participant mapping

Pattern role	Actual class / function	File and line range
Target	`AbstractInput`	`data/code/mycodo/inputs/base_input.py:18-271`
Adapter	`adafruit_i2c_soil.InputModule`	`data/code/mycodo/inputs/adafruit_i2c_soil.py:50-84`
Adaptee	`adafruit_seesaw.seesaw.Seesaw (dependency, defined outside scope, use site)`	`data/code/mycodo/inputs/adafruit_i2c_soil.py:61-67`
Adapter	`weather_openweathermap_weather.InputModule`	`data/code/mycodo/inputs/weather_openweathermap_weather.py:82-146`
Adaptee	`OpenWeatherMap HTTP API via requests.get (dependency, defined outside scope, use site)`	`data/code/mycodo/inputs/weather_openweathermap_weather.py:111-112`
Client	`InputController.update_measure`	`data/code/mycodo/controllers/controller_input.py:345-393`

Evidence: Input modules wrap foreign interfaces behind the AbstractInput get_measurement() contract: the soil module holds a Seesaw driver in self.sensor (adafruit_i2c_soil.py:65-67) and translates moisture_read()/get_temp() into channel values (79, 82); the weather module translates the OpenWeatherMap JSON from requests.get (weather_openweathermap_weather.py:111-120) into the same channel dict (131-144). InputController uses both only through AbstractInput (controller_input.py:313, 361).

Diagram: `Part1/pattern_architecture/08_adapter_sensor_driver_adaptation.mmd` (editable Mermaid) and `.png` (Figure 9).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/`

`code/` (script check of every range plus manual reading of the calls). Overlaps with Template Method 1 (the same `get_measurement` is a primitive operation); counted separately because the defining relation is translation to a foreign interface.

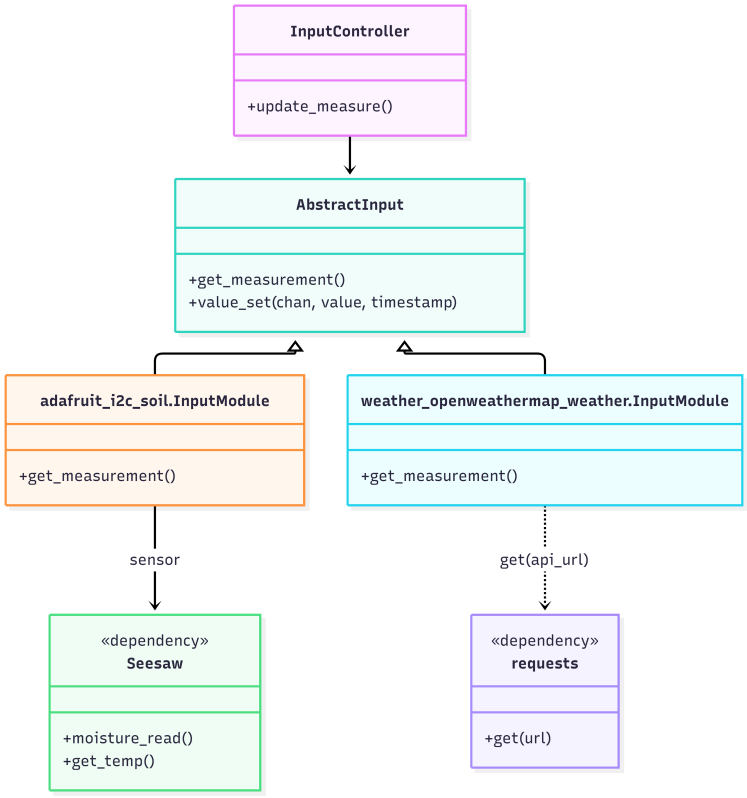


Figure 9. Adapter – Sensor Driver Adaptation.

2.3.9 Daemon RPC Client

Instance name: Daemon RPC Client

Pattern type and category: Proxy – GoF Structural

Purpose and origin: application code wrapping the Pyro5 dependency (GoF Structural).

Participant mapping

Pattern role	Actual class / function
Proxy	`DaemonControl`
ProxyConnection	`DaemonControl.proxy`
WrappedRemoteProxy	`Pyro5.api.Proxy (dependency, defined outside scope, import site)`
Client	`bang_bang.CustomModule.loop`
Client	`InputController.loop`
Client	`AbstractConditional.run_action`
Client	`AbstractOutput.check_triggers`
Client	`daemon_active`
Subject	`not separately declared`
RealSubject	`daemon object at PYRO_URI (defined outside scope)`

Source locations: `data/code/mycodo/mycodo_client.py:50-292`; `data/code/mycodo/mycodo_client.py:65-74`; `data/code/mycodo/mycodo_client.py:32-32`; `data/code/mycodo/functions/`

bang_bang.py:161-210; data/code/mycodo/controllers/controller_input.py:129-238; data/code/mycodo/controllers/base_conditional.py:40-64; data/code/mycodo/outputs/base_output.py:457-585; data/code/mycodo/mycodo_client.py:294-302

Evidence: Local stand-in for the remote daemon: DaemonControl.proxy() builds a Pyro5 Proxy with the configured URI and timeout (mycodo_client.py:65-74) and each method forwards under the same name (e.g. 126, 200-201); check_daemon adds timeout control and error translation (80-107). Daemon-side modules hold it as self.control and call it like a local object (bang_bang.py:118, 177, 195-197; controller_input.py:77, 166-171; base_conditional.py:33, 60-61; base_output.py:43, 540-541).

Diagram: Part1/pattern_architecture/09_proxy_daemon_rpc_client.mmd (editable Mermaid) and .png (Figure 10).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against data/code/ (script check of every range plus manual reading of the calls). Facade and Broker were rejected (single remote subject; broker side outside scope).

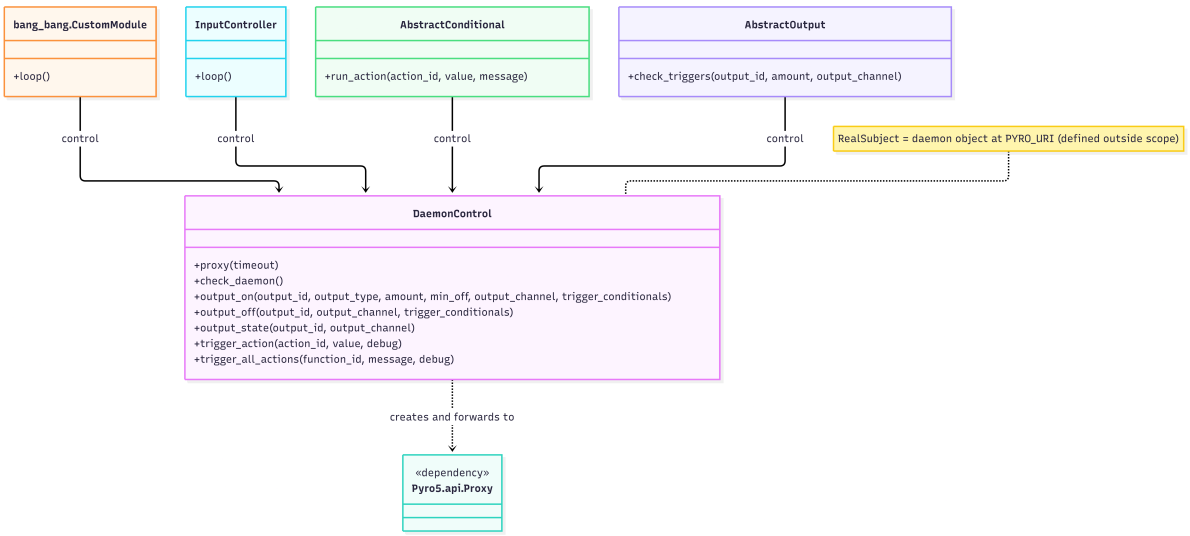


Figure 10. Proxy – Daemon RPC Client.

2.3.10 Automation/Alert Actions

Instance name: Automation/Alert Actions

Pattern type and category: Command – GoF Behavioral

Purpose and origin: application code (GoF Behavioral).

Participant mapping

Pattern role	Actual class / function
Command	AbstractFunctionAction
Execute	AbstractFunctionAction.run_action
ConcreteCommand	email.ActionModule
ConcreteCommand	output_on_off.ActionModule
Receiver	send_email (defined outside scope, call site)
Receiver	DaemonControl.output_on_off
Client	AbstractConditional.run_action
Client	AbstractOutput.check_triggers
RequestChannel	DaemonControl.trigger_action
RequestChannel	DaemonControl.trigger_all_actions

Pattern role	Actual class / function
Invoker	`defined outside scope`

Source locations: `data/code/mycodo/actions/base_action.py:25-114`; `data/code/mycodo/actions/base_action.py:77-79`; `data/code/mycodo/actions/email.py:41-98`; `data/code/mycodo/actions/output_on_off.py:68-139`; `data/code/mycodo/actions/email.py:86-88`; `data/code/mycodo/mycodo_client.py:214-222`; `data/code/mycodo/controllers/base_conditional.py:40-64`; `data/code/mycodo/outputs/base_output.py:457-585`; `data/code/mycodo/mycodo_client.py:150-153`; `data/code/mycodo/mycodo_client.py:155-157`

Evidence: Stored, parameterised requests executed later through a uniform run_action(dict_vars): each action binds its parameters from the Actions row in its constructor (email.py:48-51; output_on_off.py:78-81) and run_action calls its receiver (send_email at email.py:86-88; control.output_on_off at output_on_off.py:126-131) and returns dict_vars. Requesters know only the action ID (base_conditional.py:60-61; base_output.py:540-541); the daemon-side invoker calling run_action is outside scope.

Diagram: `Part1/pattern_architecture/10_command_automation_alert_actions.mmd` (editable Mermaid) and `.png` (Figure 11).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Chain of Responsibility was rejected; the invoker is outside scope, which is the main uncertainty.

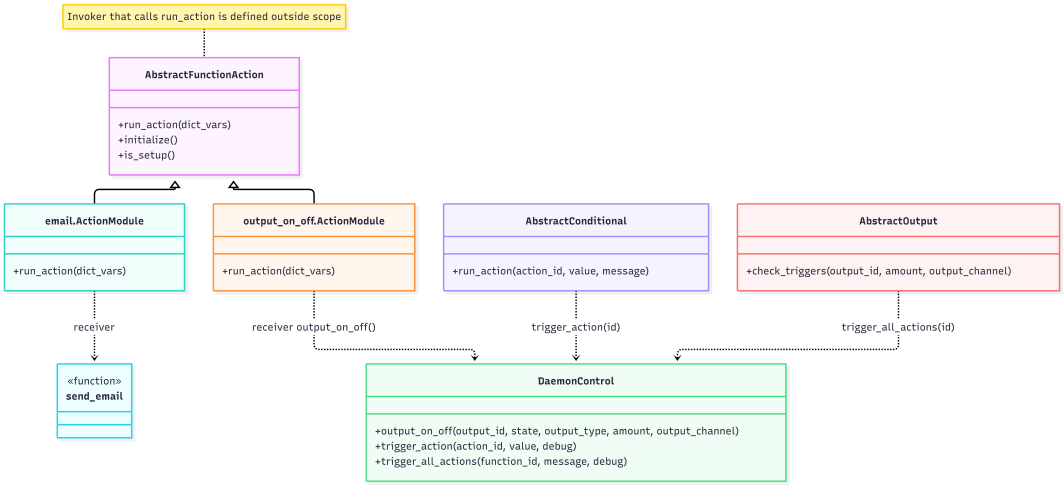


Figure 11. Command – Automation/Alert Actions.

2.3.11 Input/Output Module Loading

Instance name: Input/Output Module Loading

Pattern type and category: Plugin – Architectural

Purpose and origin: application code (PoEAA, Fowler 2002).

Participant mapping

Pattern role	Actual class / function	File and line range
PluginLoader	`load_module_from_file`	`data/code/mycodo/utils/modules.py:10-21`
PluginRegistry	`parse_input_information`	`data/code/mycodo/utils/inputs.py:163-308`

Pattern role	Actual class / function	File and line range
PluginRegistry	parse_output_information	data/code/mycodo/utils/outputs.py:33-148
RegistryQuery	output_types	data/code/mycodo/utils/outputs.py:183-189
PluginInterface	AbstractInput	data/code/mycodo/inputs/base_input.py:18-271
PluginInterface	AbstractOutput	data/code/mycodo/outputs/base_output.py:30-641
Host	InputController.initialize_variables	data/code/mycodo/controllers/controller_input.py:246-343
PluginMetadata	adafruit_i2c_soil.INPUT_INFORMATION	data/code/mycodo/inputs/adafruit_i2c_soil.py:19-47
ConcretePlugin	adafruit_i2c_soil.InputModule	data/code/mycodo/inputs/adafruit_i2c_soil.py:50-84
PluginMetadata	weather_openweathermap_weather.INPUT_INFORMATION	data/code/mycodo/inputs/weather_openweathermap_weather.py:41-79
ConcretePlugin	weather_openweathermap_weather.InputModule	data/code/mycodo/inputs/weather_openweathermap_weather.py:82-146
PluginMetadata	pump_gpio.OUTPUT_INFORMATION	data/code/mycodo/outputs/pump_gpio.py:46-142
ConcretePlugin	pump_gpio.OutputModule	data/code/mycodo/outputs/pump_gpio.py:145-353
PluginMetadata	on_off_gpio.OUTPUT_INFORMATION	data/code/mycodo/outputs/on_off_gpio.py:28-110
ConcretePlugin	on_off_gpio.OutputModule	data/code/mycodo/outputs/on_off_gpio.py:113-200

Evidence: Drop-in sensor/actuator modules: registries scan the built-in and custom directories (inputs.py:186-197; outputs.py:55-66), import each file with importlib via load_module_from_file (modules.py:14-16), keep modules exposing INPUT_INFORMATION/OUTPUT_INFORMATION and record their file_path (inputs.py:204-214; outputs.py:73-83). InputController loads the selected file and instantiates the well-known InputModule class (controller_input.py:309-313) and detects optional capabilities by reflection (327, 334).

Diagram: `Part1/pattern_architecture/11_plugin_input_output_module_loading.mmd` (editable Mermaid) and `.png` (Figure 12).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Factory Method was rejected (no creator hierarchy).

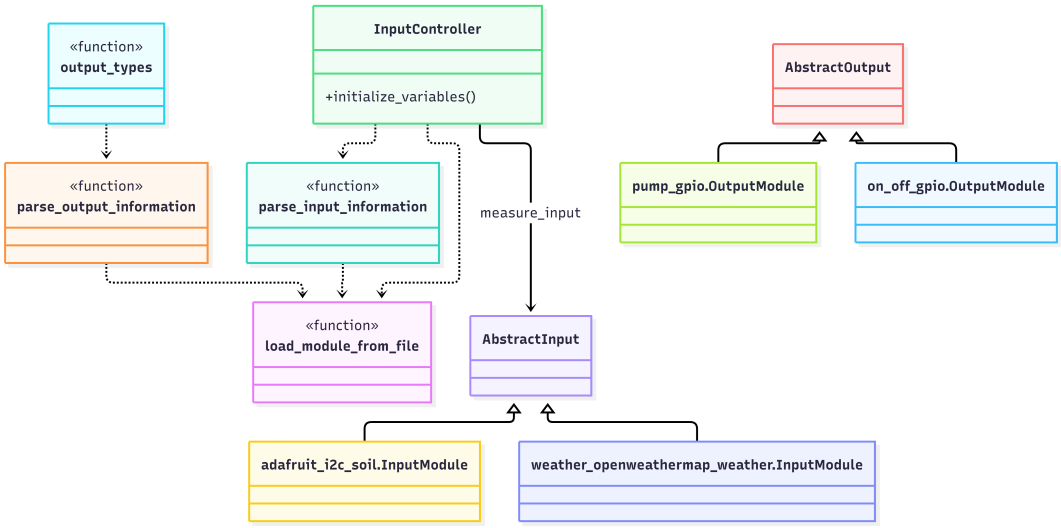


Figure 12. Plugin – Input/Output Module Loading.

2.3.12 Daemon Module Base

Instance name: Daemon Module Base

Pattern type and category: Layer Supertype – Architectural

Purpose and origin: application code (PoEAA, Fowler 2002).

Participant mapping

Pattern role	Actual class / function	File and line range
LayerSupertype	`AbstractBaseController`	`data/code/mycodo/abstract_base_controller.py:35-543`
SharedService	`AbstractBaseController.setup_custom_options`	`data/code/mycodo/abstract_base_controller.py:67-74`
SharedService	`AbstractBaseController.setup_device_measurement`	`data/code/mycodo/abstract_base_controller.py:76-88`
SharedService	`AbstractBaseController._set_custom_option`	`data/code/mycodo/abstract_base_controller.py:425-439`
LayerMember	`AbstractInput`	`data/code/mycodo/inputs/base_input.py:18-271`
LayerMember	`AbstractOutput`	`data/code/mycodo/outputs/base_output.py:30-641`
LayerMember	`AbstractController`	`data/code/mycodo/controllers/base_controller.py:20-129`
LayerMember	`AbstractFunction`	`data/code/mycodo/functions/base_function.py:17-83`
LayerMember	`AbstractFunctionAction`	`data/code/mycodo/actions/base_action.py:25-114`

Evidence: Common supertype of all daemon-side module types holding cross-cutting infrastructure (logger, option parsing, device measurements, DB-stored custom options, measurement look-ups). Members delegate to it instead of duplicating: base_input.py:43, 264-265; base_output.py:625-626; base_function.py:35, 76-77; base_controller.py:38; concrete modules call setup_custom_options (weather_openweathermap_weather.py:92; bang_bang.py:138; email.py:50) and get_last_measurement (bang_bang.py:172). AbstractConditional does not inherit it and re-implements custom options (base_conditional.py:103-126).

Diagram: `Part1/pattern_architecture/12_layer_supertype_daemon_module_base.mmd` (editable Mermaid) and `.png` (Figure 13).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). GRASP Pure Fabrication/Polymorphism were not reported separately.

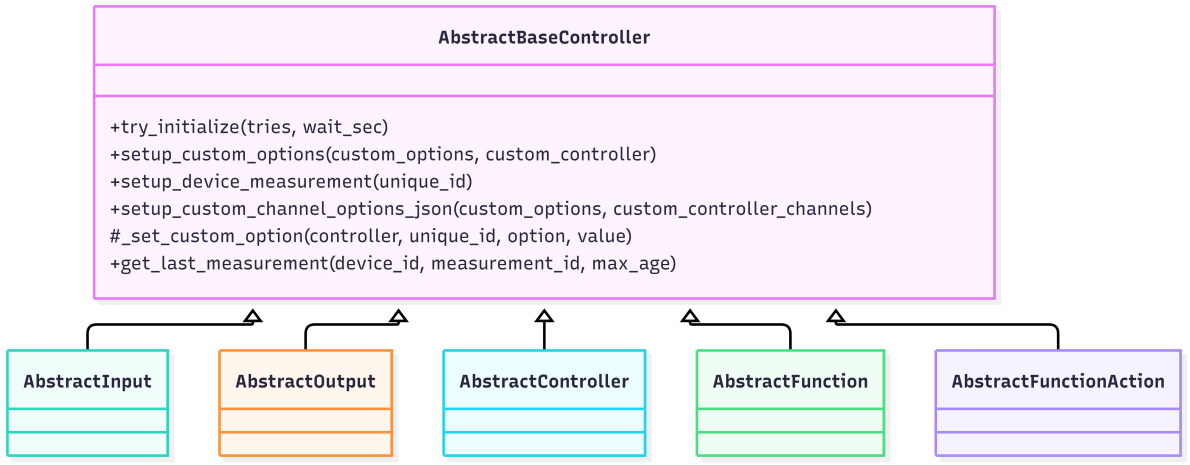


Figure 13. Layer Supertype – Daemon Module Base.

2.3.13 User and Role Persistence

Instance name: User and Role Persistence

Pattern type and category: Active Record – Architectural

Purpose and origin: application code on the Flask-SQLAlchemy framework (PoEAA, Fowler 2002).

Participant mapping

Pattern role	Actual class / function
ActiveRecordBase	`CRUDMixin`
PersistenceOperation	`CRUDMixin.save`
PersistenceOperation	`CRUDMixin.delete`
ActiveRecord	`User`
DomainLogic	`User.set_password`
DomainLogic	`User.check_password`
ActiveRecord	`Role`
ORMBase	`db.Model (framework, db defined outside scope)`

Source locations: `data/code/mycodo/databases/__init__.py:10-32`; `data/code/mycodo/databases/__init__.py:16-24`; `data/code/mycodo/databases/__init__.py:26-32`; `data/code/mycodo/databases/models/user.py:11-52`; `data/code/mycodo/databases/models/user.py:35-39`; `data/code/mycodo/databases/models/user.py:42-52`; `data/code/mycodo/databases/models/role.py:7-27`; `data/code/mycodo/databases/models/user.py:11-11`; `data/code/mycodo/databases/models/role.py:7-7`

Evidence: Table-mapped domain objects that persist themselves: User and Role (user.py:11-29; role.py:7-22) inherit save()/delete() from the application CRUDMixin, which acts on self via db.session.add/delete/commit (___init___.py:19-20, 29-30); User also carries domain logic (bcrypt password hashing at user.py:35-52). Framework side: db.Model/db.Column/db.session (db defined outside scope).

Diagram: `Part1/pattern_architecture/13_active_record_user_and_role_persistence.mmd` (editable Mermaid) and `.png` (Figure 14).

Verification: Confirmed; verified by Team B10. Roles and line ranges were checked against `data/code/` (script check of every range plus manual reading of the calls). Repository and Unit of Work were rejected (no repository abstraction; session handling outside scope).

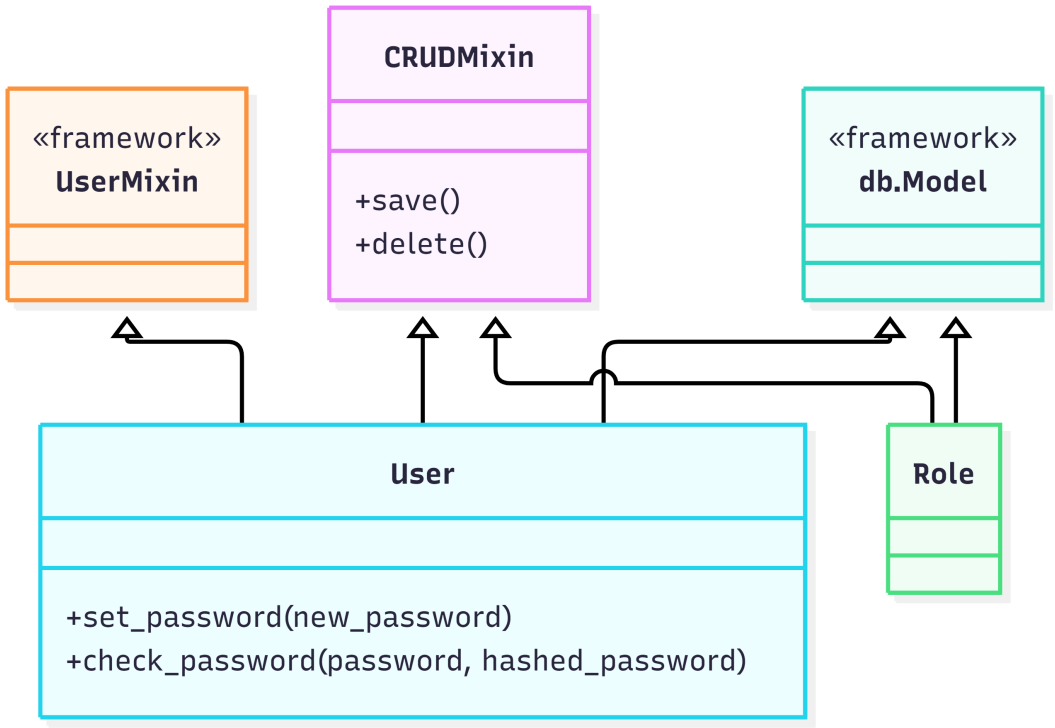


Figure 14. Active Record – User and Role Persistence.

3 Part 2 Evolution and refactoring

All three changes extend the irrigation function `mycodo/functions/bang_bang.py` (`CustomModule`, an on/off hysteresis controller that switches a pump from a soil-moisture reading). Each change has three commits: (a) characterization tests on the unchanged code, (b) a behaviour-preserving refactoring, (c) the feature with tests named after its acceptance criteria. Patches: `Part2/changes/commits/`; diffs: `Part2/changes/ChangeN/{characterization_tests,refactoring,feature}.diff`.

CH01 – Add new crop types

Change and acceptance criteria: A farmer selects a crop (tomato 65±5 %, rice 85±5 %, wheat 50±5 %) instead of typing setpoint and hysteresis; new crops are registered in one place. Acceptance: AC1–AC7 (crop values applied, two functions with different crops, `custom`/missing option keeps old behaviour, register a new crop without editing `bang_bang.py`, unknown crop logged and ignored, invalid definitions rejected, UI option lists all crops). (`Part2/changes/Change1/requirement.md`)

Before and after: Before `ee70a111c`; commits dee5d7d76, d454803d0, bdc43f9a3. Changed paths: `mycodo/functions/bang_bang.py`; new `mycodo/utils/crop_profiles.py`; tests `test_bang_bang_characterization.py`, `test_ch01_crop_profiles.py`.

Implementation and tests: 25 characterization tests (one parametrised over 15 decisions) pass

before and after (b); 42/42 after (c). Run: `cd Part2/code; python -m pytest -p no:cacheprovider -v mycodo/tests/case_study`. Log: `Part2/changes/Change1/test_result.log`; demo: `Part2/changes/Change1/demo_output.txt`.

Pattern effects: changed or new instances below; all other 11 instances preserved (`Part2/pattern_evolution.csv`, Code after Change1).

Instance	Pattern	Status	Change description
Module Initialization with Retry	Template Method	modified	Same roles and try_initialize skeleton; code inside the concrete step bang_bang.CustomModule.initialize changed: now also applies the selected crop profile via _apply_crop_profile().
Daemon RPC Client	Proxy	modified	Same roles; the client bang_bang.CustomModule.loop changed in Change1: Extract Method replaced the four setpoint +/- hysteresis expressions by _lower_threshold()/_upper_threshold(). Calls to the Proxy (output_state, output_on, output_off) are unchanged.

Refactoring: Duplicated Code – the band edges `setpoint ± hysteresis` were computed four times in `loop()` (baseline lines 188, 194, 199, 205). **Extract Method** → `_lower_threshold()` / `_upper_threshold()`; the 25 characterization tests pass unchanged (`Part2/changes/Change1/refactoring.diff`).

Before/after metrics: lizard 1.24.1 on the scope files + new modules (tests excluded): original 24 files / 4024 NLOC / 240 functions / max CCN 51 / avg 3.6; after this change 25 / 4098 / 248 / 51 / 3.5; tests 42/42 passed (`Part2/changes/metrics.csv`). Max CCN 51 is `base_output.output_on_off`, which no change touches.

CH02 – Add different irrigation strategies

Change and acceptance criteria: Irrigation can follow a policy: plain hysteresis, only inside time windows (HH:MM–HH:MM, may cross midnight), or skip while a rain indicator is above a threshold; policies stack. Acceptance: AC1–AC9 including invalid windows / missing rain measurement falling back to hysteresis. (`Part2/changes/Change2/requirement.md`)

Before and after: Before `bdc43f9a3`; commits fb1794585, c18ecb1e5, 9e3d56e8d. Changed paths: `mycodo/functions/bang_bang.py`; new `mycodo/utils/irrigation_strategies.py`; tests `test_ch02_characterization.py`, `test_ch02_irrigation_strategies.py`.

Implementation and tests: 6 new characterization tests (48 in total) pass before and after (b); 64/64 after (c). A mutation check (bypassing the time-window test) made AC2, AC3 and AC8 fail. Run: `cd Part2/code; python -m pytest -p no:cacheprovider -v mycodo/tests/case_study`. Log: `Part2/changes/Change2/test_result.log`; demo: `Part2/changes/Change2/demo_output.txt`.

Pattern effects: changed or new instances below; all other 11 instances preserved (`Part2/pattern_evolution.csv`, Code after Change2).

Instance	Pattern	Status	Change description
Module Initialization with Retry	Template Method	modified	Same roles and try_initialize skeleton; code inside the concrete step bang_bang.CustomModule.initialize changed: now also builds the irrigation strategy via _build_irrigation_strategy().
Daemon RPC Client	Proxy	modified	Same roles; the client bang_bang.CustomModule.loop changed in Change2: the direction if/elif was replaced by a strategy call; output_on/output_off are now sent from _apply_decision(). Calls to the Proxy (output_state, output_on, output_off) are unchanged.
Irrigation Decision Strategy	Strategy	newly_introduced	New collaboration: Replace Conditional with Polymorphism (step b) introduced IrrigationStrategy with RaiseHysteresisStrategy/LowerHysteresisStrategy; the feature (step c) added TimeWindowStrategy, RainSkipStrategy and build_irrigation_strategy.
Irrigation Restriction Decorators	Decorator	newly_introduced	New collaboration: IrrigationRestriction wraps any IrrigationStrategy; TimeWindowStrategy and RainSkipStrategy add irrigation restrictions.

Refactoring: Switch Statements – `loop()` switched on the type code `self.direction` with mirrored raise/lower branches. **Replace Conditional with Polymorphism** → `IrrigationStrategy.decide()` with `RaiseHysteresisStrategy` / `LowerHysteresisStrategy`; 48 tests pass unchanged. `loop()` CCN 14 → 7.

Before/after metrics: lizard 1.24.1 on the scope files + new modules (tests excluded): original 24 files / 4024 NLOC / 240 functions / max CCN 51 / avg 3.6; after this change 26 / 4298 / 268 / 51 / 3.5; tests 64/64 passed (`Part2/changes/metrics.csv`). Max CCN 51 is `base_output.output_on_off`, which no change touches.

CH03 – Add sensor-based alerts

Change and acceptance criteria: Alert the farmer when the soil is critically dry or the sensor stops reporting; alerts are logged and optionally e-mailed, once per episode, and re-armed on recovery. Acceptance: AC1–AC9 (critical-dry alert, missing reading turns the pump off and alerts, a `None`/`[]` reading no longer crashes, failing listener isolated, recipients parsed). (`Part2/changes/Change3/requirement.md`)

Before and after: Before `9e3d56e8d`; commits 1146fe42a, 349ed087c, bce41febb. Changed paths: `mycodo/functions/bang_bang.py`; new `mycodo/utils/sensor_alerts.py`; tests `test_ch03_characterizatio` `py`, `test_ch03_sensor_alerts.py`.

Implementation and tests: 8 new characterization tests (72 in total) pass before and after (b); 83/83 after (c). Disabling `_notify` made 7 AC tests fail. Run: `cd Part2/code; python -m pytest -p no:cacheprovider -v mycodo/tests/case_study`. Log: `Part2/changes/Change3/test_result.log`; demo: `Part2/changes/Change3/demo_output.txt`.

Pattern effects: changed or new instances below; all other 12 instances preserved (`Part2/pattern_evolution.csv`, Code after Change3).

Instance	Pattern	Status	Change description
Module Initialization with Retry	Template Method	modified	Same roles and try_initialize skeleton; code inside the concrete step bang_bang.CustomModule.initialize changed: now also attaches the alert listeners via _setup_alert_listeners().
Daemon RPC Client	Proxy	modified	Same roles; the client bang_bang.CustomModule.loop changed in Change3: Extract Method (_period_elapsed, _read_measurement, _handle_missing_measurement); output_off for a missing value is sent from _handle_missing_measurement(), which also raises an alert. Calls to the Proxy (output_state, output_on, output_off) are unchanged.
Irrigation Decision Strategy	Strategy	modified	Same roles; the Context code bang_bang.CustomModule.loop changed (Extract Method, safe measurement value) and moved. Strategy classes unchanged.
Sensor Alert Notification	Observer	newly_introduced	New collaboration: SensorAlertNotifier notifies LogAlertListener and EmailAlertListener of missing-measurement and critically-dry alerts.

Refactoring: Long Method – `loop()` mixed timer gating, reading, missing-value handling and strategy evaluation. **Extract Method** → `_period_elapsed()`, `_read_measurement()`, `_handle_missing_measurement()`; 72 tests pass unchanged. `loop()` CCN 7 → 5.

Before/after metrics: lizard 1.24.1 on the scope files + new modules (tests excluded): original 24 files / 4024 NLOC / 240 functions / max CCN 51 / avg 3.6; after this change 27 / 4426 / 288 / 51 / 3.3; tests 83/83 passed (`Part2/changes/metrics.csv`). Max CCN 51 is `base_output.output_on_off`, which no change touches.

3.1 Updated pattern counts

Snapshot	Pattern types	Instances	Concrete participant classes	Rejected / uncertain
Original code	8	13	18	23 / 5
Code after Change1	8	13	18	23 / 5
Code after Change2	9	15	22	23 / 5
Code after Change3	10	16	24	23 / 5

Concrete participant classes are the distinct symbols in a concrete role (ConcreteClass, ConcreteStrategy, ConcreteDecorator, ConcreteObserver, Adapter, Proxy, ConcreteCommand, ConcretePlugin, ActiveRecord) in `Part2/pattern_evolution.csv`, excluding the five abstract Layer Supertype members (as in 2.1). No instance was removed or replaced; continuing collaborations keep their Part 1 names. New collaborations: CH02 *Irrigation Decision Strategy* (Strategy) and *Irrigation Restriction Decorators* (Decorator, wrapping any strategy); CH03 *Sensor Alert Notification* (Observer). The Part 1 uncertain candidate "Observer – output trigger notification" is unrelated and unchanged.

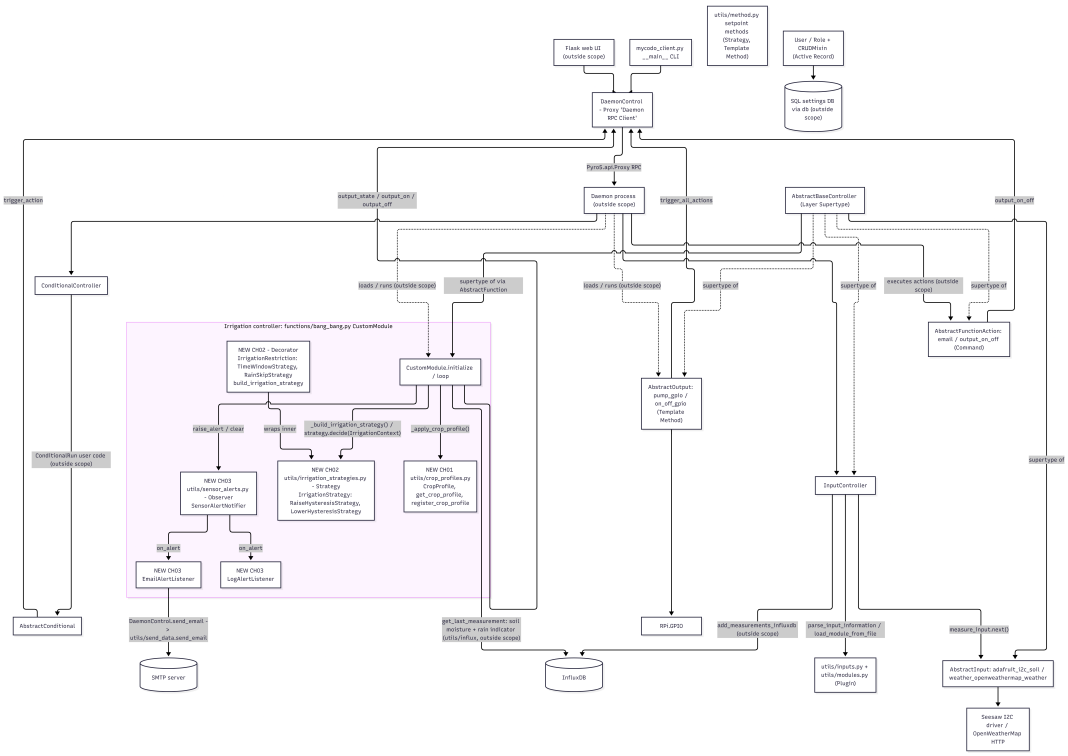


Figure 15. Architecture after CH03 (`Part2/diagrams/evolved_architecture.mmd`); per-change UML in `Part2/diagrams/`.

Interpretation: NLOC grew from 4024 to 4426 (+10 %) because three modules were added, while `bang_bang.loop()` fell from CCN 14 to 5 and each new policy or listener is a small class that can be tested alone (CCN ≤ 6). New crops, policies and alert channels are added without editing the controller (open/closed). Trade-offs: more classes and indirection to read; Strategy and Decorator over the same classes could be seen as one extended instance; alerts are edge-triggered with in-memory state, so a restart re-arms them; the tests mock the database, the daemon and InfluxDB, so end-to-end behaviour in a running Mycodo was not verified. More or fewer LOC alone does not prove a better design.

4 Part 3 Cross-language comparison

Selected instances: One verified instance per confirmed pattern type (8 types $\times$ 4 languages = 32 implementations):

Pattern	Instance	Selection reason
Template Method	Date/Daily Setpoint Interpolation	Pure computation; all roles are real classes (the other five instances need hardware, threads or numpy).
Strategy	Setpoint Schedule Calculation	Only Strategy instance; sine / duration / cascade are pure time→setpoint functions.
Adapter	Sensor Driver Adaptation	Only Adapter instance; Seesaw driver and weather HTTP replaced by in-memory adaptees with the same method names.
Proxy	Daemon RPC Client	Only Proxy instance; Pyro5 and the daemon replaced by an in-memory remote proxy.
Command	Automation/Alert Actions	Only Command instance; receivers and the out-of-scope invoker replaced by stand-ins.
Plugin	Input/Output Module Loading	Only Plugin instance; importlib replaced by an in-memory module catalogue.

Pattern	Instance	Selection reason
Layer Supertype	Daemon Module Base	Only Layer Supertype instance; option parsing is pure logic.
Active Record	User and Role Persistence	Only Active Record instance; in-memory session, std-lib stand-in for bcrypt (not secure).

Common specifications: `Part3/test-cases/SPEC.md` defines inputs, outputs, rules and error cases per problem; `Part3/test-cases/<problem>_cases.csv` holds 9–12 shared cases each (at least two ERROR cases). All runners read the same CSV and compare tokens with a numeric tolerance of 1e-9; an ERROR case passes when the production code raises.

Implementations: `Part3/<pattern_problem>/{java,python,javascript,cpp}/`: one production file, one separate test runner and a README with the working folder and exact commands. Toolchains: OpenJDK 21.0.2, CPython 3.12.7, Node.js v23.11.0, MinGW g++ 6.3.0 `-std=c++14 -O2` (`Part3/evidence/toolchains.txt`).

Actual results: All 32 runners pass (Template Method, Strategy, Adapter, Proxy, Command, Active Record 12/12; Layer Supertype 11/11; Plugin 9/9). Logs: `Part3/evidence/tests_*.txt`; re-run 2026-10-25: `data/test_result/part3_summary.txt`.

Static and dynamic comparison: recorded in `Part3/llm/pattern_crosslanguage.csv` (runtime/compiler, test command and language notes per row); the measured values are:

Pattern	Language	Source LOC	Max CC	Tests	Median ms
Template Method	Java	77	7	12/12	157.771
Template Method	Python	37	6	12/12	72.495
Template Method	JavaScript	54	11	12/12	90.926
Template Method	C++	85	21	12/12	33.060
Strategy	Java	106	9	12/12	156.784
Strategy	Python	67	10	12/12	73.589
Strategy	JavaScript	83	9	12/12	96.586
Strategy	C++	97	12	12/12	31.403
Adapter	Java	108	7	12/12	156.345
Adapter	Python	80	7	12/12	72.979
Adapter	JavaScript	88	7	12/12	92.718
Adapter	C++	106	7	12/12	32.800
Proxy	Java	111	7	12/12	158.345
Proxy	Python	60	6	12/12	70.879
Proxy	JavaScript	69	5	12/12	88.559
Proxy	C++	103	7	12/12	33.030
Command	Java	112	8	12/12	173.109
Command	Python	67	7	12/12	74.217
Command	JavaScript	88	9	12/12	90.155
Command	C++	130	8	12/12	32.462
Plugin	Java	87	9	9/9	156.021
Plugin	Python	61	8	9/9	65.257
Plugin	JavaScript	75	9	9/9	91.112
Plugin	C++	94	8	9/9	32.298
Layer Supertype	Java	68	23	11/11	156.814
Layer Supertype	Python	56	19	11/11	65.902
Layer Supertype	JavaScript	65	22	11/11	94.884
Layer Supertype	C++	82	25	11/11	32.492
Active Record	Java	113	14	12/12	235.049
Active Record	Python	69	13	12/12	86.483
Active Record	JavaScript	83	7	12/12	95.829
Active Record	C++	119	14	12/12	33.528

LOC = lizard NLOC of the production file; Max CC = highest lizard CCN of a function in it; median ms = median of 5 measured whole-process runs after compiling and one warmup (start-up + reading the CSV + all cases).

Comparison: The participant structure is the same in all four languages; what moves is where the contract is checked. Java and C++ declare roles as types (abstract classes, interfaces, `final`, pure virtual functions, `override`) so a missing role method is a compile error; Python and JavaScript rely on convention and fail at run time. Inheritance is used for Template Method, Layer Supertype and the strategy/command/plugin hierarchies; composition for the Adapter adaptee, the Proxy connection, CascadeMethod's linked strategies and the Active Record session. The Proxy shows the largest contrast: Python `__getattr__` and the JavaScript `Proxy` object forward any method (60 and 69 NLOC) while Java and C++ repeat every method of the subject interface (111 and 103 NLOC). Active Record needs generic column access, free in Python (`getattr`) and JavaScript (`obj[column]`), but an explicit `column(name)` method in Java and C++. Static typing mainly costs size: Java 68–113 and C++ 82–130 NLOC versus Python 37–80 and JavaScript 54–88, while max CC is usually within a few points. The largest CC differences come from missing library support (hand-written date parsing: JavaScript CC 11, C++ CC 21 vs Python 6) and from Layer Supertype's option type-switch (19–25 in every language). Medians were C++ 31–34 ms, Python 65–86 ms, JavaScript 89–97 ms and Java 156–235 ms; with 9–12 tiny cases these measure runtime start-up, not pattern speed.

Technology discussion: The patterns keep their intent across languages, as described by Gamma et al. (Template Method, Strategy, Adapter, Proxy, Command) and Fowler (Plugin, Layer Supertype, Active Record). Java (JLS SE 21, ch. 8–9) and C++ (virtual functions, `std::unique_ptr` ownership) enforce the roles at compile time and make ownership explicit; Python can enforce abstract methods with `abc` but the snippets follow the source and use plain base classes; ECMAScript classes have no abstract or final members (MDN "Classes"), while its `Proxy` object (MDN "Proxy") and Python's data model make forwarding proxies and plugin catalogues shorter. Language pitfalls found on the way (CSV splitting, number formatting, integer division, `Number(") = 0` in JavaScript, 32-bit FNV hashing) are recorded in `Part3/llm/output.txt`, section J.

5 LLM records and human verification

Run and model: Claude Opus 5.5 (`claude-opus-5-5`) in Claude Code (VS Code extension). Part 1 and Part 2: 2026-10-11; Part 3: 2026-10-12. One recorded run per part (Part 2 had a discarded first run). Each member also wrote their own prompt per part (`part*_prompt_<name>.txt`).

Inputs and outputs: Prompts: `Part1/llm/part1_prompt_Johan.txt`, `Part2/llm/part2_prompt_Johan.txt`, `Part3/llm/part3_prompt_Johan.txt`. Supplied code: `data/code/` at `ee70a111` (files listed in `Part2/llm/input.txt`, `Part3/llm/input.txt`). Raw outputs: `Part1/llm/output.txt`, `Part2/llm/output.txt`, `Part3/llm/output.txt` (raw records keep the folder names of the time: `Mycodo/` = `data/code/`, `Part2/work/` = `Part2/code/`).

Claims and evaluation: one row per output in `Part1/llm.csv`, `Part2/llm.csv` and `Part3/llm/llm.csv`; every row was verified by Team B10 against its verification_file evidence:

Part	Output	Evaluation	Correction / evidence
Pattern discovery	`Part1/llm/output.txt`	Partial	Claims: 8 pattern types / 13 instances (Template Method x6; Strategy; Adapter; Proxy; Command; Plugin; Layer Supertype; Active Record), 5 uncertain and 23 rejected candidates. All 13 instances accepted after checking each role against the source. Corrections: the section 7 sub-totals (methods 41, functions 7) were mislabelled and fixed; Command is kept although its invoker is outside scope; the six-way Template Method split and the Adapter/Template Method overlap are judgement calls explained in the report.
Pattern discovery	`Part1/patterns.csv`	Correct after minor correction	13 rows copied from section 5 of the raw output; punctuation in participant_roles was edited in 6 rows. A script confirmed that all 112 path:line references exist and lie inside their files.
Pattern discovery	`Part1/Pattern_count.csv`	Correct	Recounting Part1/patterns.csv gives 8 types and 13 instances, matching every per_pattern row.
Evolution	not kept (first run discarded)	Cannot confirm	the output of this first run no longer exists; its CH01 design was reproduced unchanged by PROMPT 2.
Evolution	`Part2/llm/output.txt`	Partial	CH01-CH03 implemented with 9 commits (characterization tests, refactoring, feature per change); 83/83 case-study tests pass and were re-run on 2026-10-25 after the folder move. Smells/refactorings: Duplicated Code / Switch Statements / Long Method with Extract Method / Replace Conditional with Polymorphism / Extract Method. Corrections: upstream Mycodo tests were not run (Windows; DB, daemon and hardware are mocked); "modified" for the Proxy and counting Strategy + Decorator separately are judgement calls.
Evolution	`Part2/pattern_evolution.csv`	Correct	57 rows (13 baseline + 13/15/16); later ranges were computed with Python ast by Part2/tools/build_evolution_csv.py. Re-running the tool on 2026-10-25 reproduced the file byte for byte, and all 502 path:line references lie inside their files.
Evolution	`Part2/Pattern_count.csv`	Correct	Recounting the evolution rows (removed rows excluded; none removed) gives 13/8, 13/8, 15/9 and 16/10 instances/types for Original, Change1, Change2 and Change3.
Evolution	`Part2/changes/metrics.csv`	Correct	Every value is extracted by Part2/tools/lizard_metrics.py from the raw lizard output; test counts come from the step-c pytest logs.
Cross-language analysis	`Part3/llm/output.txt`	Partial	All 8 confirmed pattern types implemented in Java, Python, JavaScript and C++; 32/32 test runners pass (re-run 2026-10-25). Corrections: Java (68-113 NLOC) and C++ (82-130) files exceed the "about 30-90" guideline; log-and-abort errors in the source became exceptions; DB, hardware, network, Pyro5 and bcrypt are replaced by stand-ins (the bcrypt stand-in is not secure); timings mostly measure process start-up.

Part	Output	Evaluation	Correction / evidence
Cross-language analysis	`Part3/llm/pattern_crosslanguage.csv`	Correct after a format correction	32 rows (8 x 4); every number is rebuilt by Part3/tools/collect_results.py from the raw lizard, test and timing evidence and matched the LLM file. Correction: the language column held folder names (java, cpp); it now uses Java, Python, JavaScript and C++ as the spec requires. An earlier lizard pass that skipped the .hpp files was fixed and re-measured before the CSV was built.
Cross-language analysis	`Part3/test-cases/SPEC.md`	Partial	8 problems with 9-12 cases each (at least 2 error cases); non-trivial expected values (dew point, sine, FNV hash) were computed independently from the source formulas. Cannot confirm: equivalence with Mycodo for untested edge cases (e.g. Duration modulo an empty cycle).

Verification evidence for every row is in the `verification_file` column of `Part1/llm.csv`, `Part2/llm.csv` and `Part3/llm/llm.csv`. Generated CSVs were re-created from raw tool output by `Part2/tools/build_evolution_csv.py` and `Part3/tools/collect_results.py`; both reproduced the submitted values.

Generated code: The Part 2 code (three modules, `bang_bang.py` edits, 83 case-study tests) and the 64 Part 3 files were generated by the LLM and accepted only after the tests passed; mutation checks in Part 2 showed the new tests fail when the feature is disabled. All tests were re-run on 2026-10-25 after reorganising the folder.

6 Findings, contributions and references

Findings:

- Mycodo's extensibility rests on a few base classes: 8 pattern types / 13 instances at the baseline, dominated by Template Method (6 instances) around `AbstractBaseController` and its input/output/controller subclasses, plus Plugin loading of device modules.
- The three agriculture changes fitted the existing structure and added Strategy, Decorator and Observer (16 instances / 10 types after CH03) while keeping all 13 baseline collaborations; the refactorings cut `bang_bang.loop()` from CCN 14 to 5.
- Across Java, Python, JavaScript and C++ the same roles were reproduced with identical test results (32/32); static typing costs size and moves error detection to compile time, dynamic features shorten Proxy, Plugin and Active Record.
- Limitations: static analysis of 24 files only; Command's invoker and the daemon are out of scope; Part 2 tests use mocks; Part 3 timings measure process start-up; several classifications (six Template Method instances, Strategy vs Decorator) are judgement calls.

Contributions: All four members carried out Part 1, Part 2 and Part 3 independently with LLMs, each using their own prompts. The individual results were then compared, verified against the source and combined into one team result: the CSVs, code, tests and evidence in this folder.

Roll number	Name	Batch	Contribution and evidence
AM.SC.U4AIE23038	Johan Rajesh	AIE	Parts 1, 2 and 3 with an LLM using own prompts (`*_prompt_Johan.txt`); results compared and combined into the team result; this run's outputs are the recorded LLM records (`llm.csv`).
AM.SC.U4AIE23003	Aadarsh Jayan Nair	AIE	Parts 1, 2 and 3 with an LLM using own prompts (`*_prompt_Aadarsh.txt`); results compared and combined into the team result.
AM.SC.U4AIE23029	Dano Aby Alex	AIE	Parts 1, 2 and 3 with an LLM using own prompts (`*_prompt_Dano.txt`); results compared and combined into the team result, including the cross-check of the 13 Part 1 instances.
AM.SC.U4AIE23140	P Arya	AIE	Parts 1, 2 and 3 with an LLM using own prompts (`*_prompt_Arya.txt`); results compared and combined into the team result.

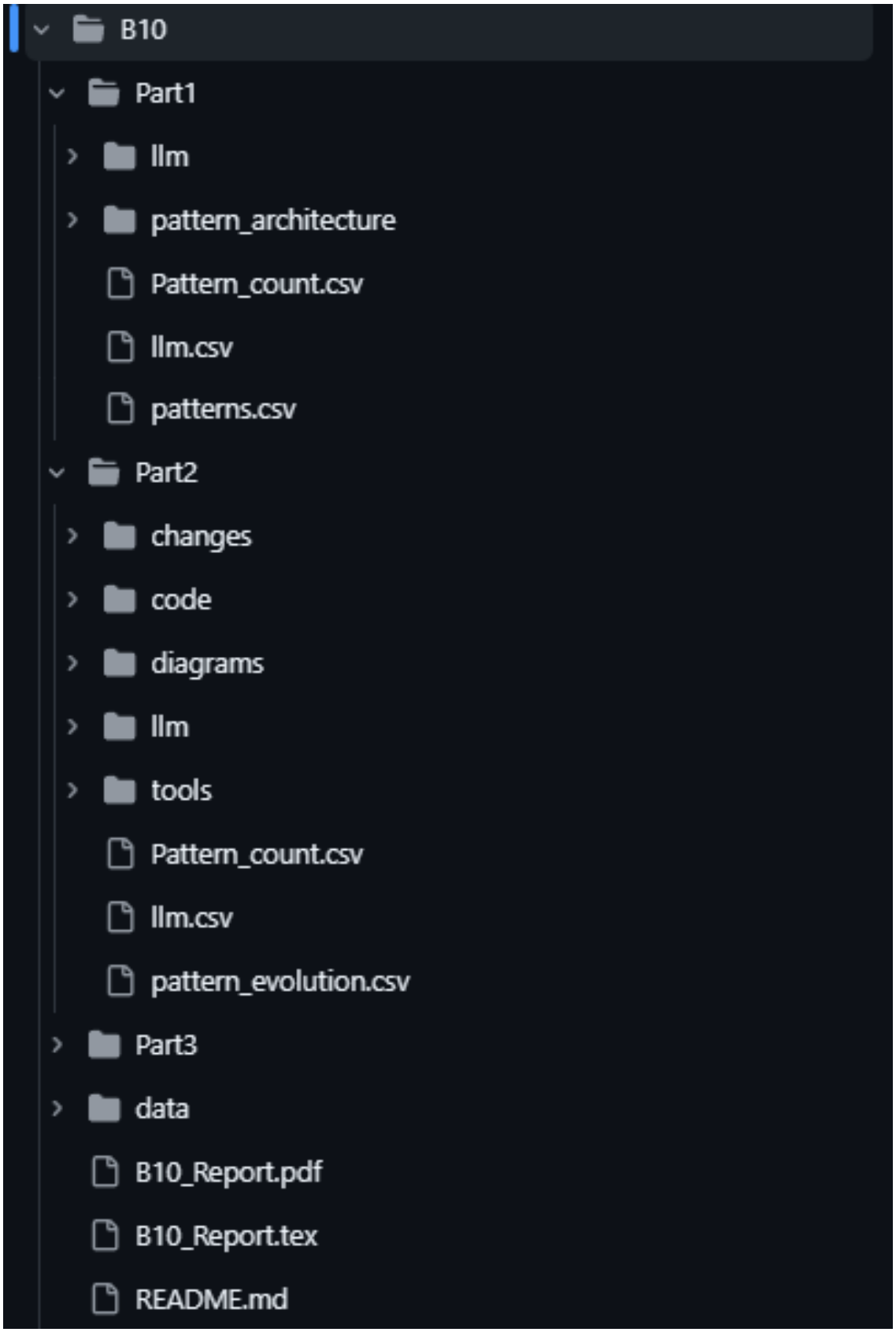
References:

- Mycodo, K. Gabriel (kizniche) and contributors, `https://github.com/kizniche/Mycodo`, v8.17.0, commit ee70a111, GPL-3.0.
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2018.
- The Java Language Specification, Java SE 21 Edition, Oracle; Python 3.12 documentation (`abc`, Data model); MDN Web Docs (Classes, Proxy); cppreference.com (virtual function specifier, `std::unique_ptr`).
- Tools: lizard 1.24.1 (`https://github.com/terryyin/lizard`), pytest 9.0.2, Mermaid (diagrams), Claude Opus 5.5 via Claude Code.

7 Team folder and datasets (screenshot from the GitHub repository)

Team folder `B10/` in 23CSE455_DP_CaseStudy (structure of DP_Csestudy.pdf):

```
B10/
|-- README.md, B10_Report.pdf, B10_Report.tex
|-- data/code/                 original Mycodo source (ee70a111)
|-- data/test_result/          test logs + captures, run_all_tests.py
|-- Part1/  patterns.csv, Pattern_count.csv, pattern_architecture/ (.mmd + .png),
|           llm/ (part1_prompt_<name>.txt, input.txt, output.txt, run_info.txt), llm.csv
|-- Part2/  code/, changes/Change1-3/ (requirement, diffs, test_result.log), changes/commits/,
|           pattern_evolution.csv, Pattern_count.csv, diagrams/, llm/, tools/, llm.csv
`-- Part3/  <pattern_problem>/{java,python,javascript,cpp}/ (snippet, runner, README) x 8,
            test-cases/, evidence/, tools/,
            llm/ (part3_prompt_<name>.txt, llm.csv, pattern_crosslanguage.csv, B10_slide.pdf)
```



Team folder `B10/` in the GitHub repository 23CSE455_DP_CaseStudy.